

Redspher — Full Stack Developer

TypeScript · React · Node.js · AWS · REST · CI/CD · AI/LLM/MCP | Question & answer knowledge base

Answer-first knowledge base for the technical round. Each item is a question you can plausibly be asked plus a tight, defensible answer. **Red-bordered** = highest-likelihood given the stack. **Amber notes** = the follow-up or trap the interviewer usually chases.

TypeScript

CORE FOCUS

"Strong proficiency in TypeScript" is called out twice — expect depth, not syntax trivia.

Q interface vs type — when do you reach for each?

Both describe object shapes. **interface** is open — it supports declaration merging and **extends**, so it's the default for public/object contracts and anything a consumer might augment. **type** is a closed alias but far more expressive: unions, intersections, tuples, mapped and conditional types, primitives.

Rule of thumb: **interface** for object contracts you may extend; **type** when you need unions or type-level computation.

Follow-up: "Can two interfaces with the same name coexist?" Yes — they merge. Two **type**s with the same name are a compile error. That merging is exactly why libraries expose interfaces.

Q Explain generics with a real constraint. What does `<T extends ...>` buy you?

Generics keep a function polymorphic while preserving the caller's exact type instead of collapsing to **any**. A constraint narrows what **T** may be so you can safely touch its members.

```
function prop<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

K extends keyof T guarantees the key exists and the return type is the precise value type — a typo in the key is a compile error.

Q unknown vs any vs never ?

- **any** — opts out of checking entirely; it's contagious and defeats the point. Avoid.
- **unknown** — top type: accepts anything but forces you to narrow before use. The safe replacement for **any** at boundaries (API responses, `JSON.parse`).
- **never** — bottom type: no value. Return type of a function that always throws, and the tool for exhaustiveness checks in a **switch**.

Q How do you guarantee a **switch** over a union is exhaustive?

Assign the default branch to **never**. If someone adds a union member and forgets a case, the assignment stops compiling.

```
default: {
  const _exhaustive: never = value;
  throw new Error(`Unhandled: ${_exhaustive}`);
}
```

Q What are utility types and which do you actually use?

Built-in mapped types that transform existing types: **Partial<T>** (all optional — patch payloads), **Pick / Omit** (derive DTOs from a model), **Required**, **Readonly**, **Record<K,V>**, and **ReturnType / Parameters** for inferring off functions. The value: one source of truth — derive variants instead of duplicating shapes.

Q Type guards and discriminated unions?

A **discriminated union** gives each variant a shared literal field (`kind: 'a' | 'b'`); checking it narrows the whole object. A **user-defined type guard** is a function returning `x is Foo` that teaches the compiler a runtime check. Together they turn runtime validation into compile-time safety.

React

CORE FOCUS

Expect hooks internals, re-render reasoning, and "why is this slow / buggy" scenarios.

Q `useMemo` vs `useCallback` vs `React.memo` — what problem does each solve, and when is it pointless?

- **`useMemo`** — memoizes a *computed value* across renders.
- **`useCallback`** — memoizes a *function identity*; `useCallback(fn, d) ≡ useMemo(() => fn, d)`.
- **`React.memo`** — memoizes a *component*, skipping re-render when props are shallow-equal.

They only matter together: a memoized child (`React.memo`) still re-renders if you hand it a fresh function/object each render — hence `useCallback` / `useMemo` to keep prop identity stable.

Trap: Don't sprinkle them everywhere. Memoization has its own cost, and for cheap values it's net-negative. Reach for it on expensive compute or to stabilize props into memoized children — not by default.

Q What actually triggers a re-render? Why does a component re-render when "nothing changed"?

A component re-renders when: its **state** changes, its **parent** re-renders, or a **consumed context** value changes. The usual "nothing changed" culprit is the parent re-rendering and passing a **new object/array/function reference** each time — referentially unequal even when structurally identical.

Re-render ≠ DOM update. React reconciles the virtual tree and only touches the DOM where it diverged; a re-render can produce zero DOM writes.

Q Rules of Hooks — what are they and why do they exist?

Call hooks only at the **top level** (never in conditionals, loops, or nested functions) and only from **React functions**. React tracks hook state by *call order*, not by name — a conditional hook shifts the order between renders and the wrong state gets returned.

Q The `useEffect` dependency array — behavior for `[]`, `[dep]`, and omitted?

- **Omitted** — runs after *every* render.
- `[]` — runs once after mount (cleanup on unmount).
- `[dep]` — runs on mount and whenever `dep` changes by `Object.is`.

The cleanup function runs before the next effect and on unmount — that's how you cancel subscriptions, timers, or in-flight fetches.

Trap: Lying about deps to silence the linter causes stale closures — the effect captures old state/props. Fix the real cause (functional updates, `useRef`, or moving the value in) rather than trimming the array.

Q Controlled vs uncontrolled components?

Controlled: React state is the single source of truth — `value` + `onChange`. Predictable, easy to validate/transform.

Uncontrolled: the DOM holds the value, read via a `ref`. Less code, handy for simple or file inputs. Default to controlled for anything you validate or react to as the user types.

Q Keys in lists — why not the array index?

Keys let React match elements to instances across renders. An **index** key breaks on reorder/insert/delete: React reuses the wrong instance, so input state and focus attach to the wrong row. Use a stable unique id from the data.

Q How would you debug a slow list / laggy render?

- **Profiler** (React DevTools) to find which components render and how often.
- **Virtualize** long lists so only visible rows mount.
- Stabilize props / split components so re-renders stay local.
- Memoize genuinely expensive computations.

Lead with measurement — never optimize blind.

Node.js & Backend

CORE FOCUS

You'll build "backend services" behind the React app — expect the event loop and async.

Q Node is single-threaded — how does it handle thousands of concurrent requests?

One thread runs your JavaScript, but I/O is **non-blocking**. The **event loop** hands I/O (network, disk) to the OS or libuv's thread pool and continues; when the OS signals completion, the callback is queued and runs. So the single thread is never blocked *waiting* — it's only busy while executing JS. This is why Node excels at I/O-bound workloads and struggles with CPU-bound ones (which do block the loop — offload those to worker threads or a separate service).

Q Callbacks → Promises → async/await. What does **async/await** actually do?

It's syntactic sugar over Promises. **await** pauses the async function until the Promise settles *without blocking the event loop* — other work runs meanwhile. It flattens "callback hell" into linear code and lets you use **try/catch** for async errors.

Follow-up: Sequential vs parallel awaits. Awaiting in a loop serializes independent calls; fire them together and **await Promise.all([...])** when they don't depend on each other.

Q How do you handle errors in async Express/Node code?

try/catch around awaited calls; forward to centralized error middleware via **next(err)** (or a wrapper that catches rejections). Crucially: an unhandled Promise rejection or a throw in a bare callback can take the process down — every async path needs a catch, and a top-level **unhandledRejection** handler is the safety net.

Q What is middleware and how does the chain work?

Functions with **(req, res, next)** that run in registration order. Each either responds or calls **next()** to pass control. It's how you layer cross-cutting concerns — auth, logging, body parsing, validation, error handling — without polluting route logic. Order matters: auth before the protected handler, error middleware last.

Q **process.nextTick** vs **setImmediate** vs **setTimeout(fn, 0)** ?

process.nextTick fires before the event loop continues (highest priority — starves the loop if abused). **setImmediate** runs on the check phase, after I/O. **setTimeout(fn, 0)** runs on the timers phase with a minimum delay. Knowing the ordering signals you understand the loop phases.

REST API Design

Explicitly in the stack — expect design judgment, not memorized status codes.

Q What makes an API RESTful? Design one for an **orders** resource.

Resource-oriented URLs (nouns), HTTP verbs for actions, stateless requests, correct status codes.

- **GET /orders** — list · **GET /orders/:id** — one
- **POST /orders** — create (→ **201** + **Location**)
- **PUT /orders/:id** — full replace · **PATCH** — partial
- **DELETE /orders/:id** — remove

Verbs never go in the path (**/getOrders** is wrong); nesting shows relationships: **GET /orders/:id/items** .

Q PUT vs PATCH , and what is idempotency?

PUT replaces the whole resource; **PATCH** applies a partial update. **Idempotent** = the same request repeated leaves the same server state. **GET** , **PUT** , **DELETE** are idempotent; **POST** is not (two POSTs create two rows). This matters for safe client retries on flaky networks.

Q Status codes — the ones you'll be expected to place correctly.

- **200** OK · **201** Created · **204** No Content
- **400** Bad Request · **401** Unauthenticated · **403** Authenticated-but-forbidden · **404** Not Found · **409** Conflict · **422** Validation failed
- **500** server fault · **503** unavailable

Classic trap: 401 vs 403 — 401 = "I don't know who you are," 403 = "I know, and you still can't."

Q How do you version and paginate an API?

Versioning: URL prefix (`/v1/`) is most common and cache-friendly; header-based is cleaner but less visible. **Pagination:** offset/limit is simple but drifts on inserts; **cursor-based** (keyset) is stable and scales — preferred for large or live datasets.

AWS Cloud

"Work in an AWS cloud environment" — expect service-selection reasoning. Your Solutions Architect / Developer Associate certs live here.

Q Walk through hosting this app on AWS. Which services, and why?

- **Frontend (React):** static build in **S3** behind **CloudFront** (CDN, TLS, caching).
- **Backend:** **Lambda + API Gateway** for event-driven/spiky load, or **ECS/Fargate** for long-running containers.
- **Data:** **RDS** (relational) or **DynamoDB** (key-value, serverless scale).
- **Auth:** **Cognito**. **Secrets:** Secrets Manager / SSM. **Async:** SQS/SNS/EventBridge.

Frame it as trade-offs — serverless (no ops, cold starts) vs containers (control, steadier latency).

Q Lambda vs EC2 vs Fargate — how do you choose?

Lambda: event-driven, sub-15-min tasks, scale-to-zero, pay-per-invocation — watch cold starts and duration limits.

Fargate: containers without managing servers — steady services, no cold start. **EC2:** full control / specialized needs, but you own patching and scaling. Decision axis: request pattern (spiky → Lambda, steady → Fargate) and how much infra control you need.

Q What's a cold start and how do you reduce it?

The first-invocation latency when Lambda spins up a new execution environment (init runtime + your code). Mitigate with **provisioned concurrency**, smaller bundles/fewer deps, and lighter runtimes. Relevant because it's the top gotcha of a serverless React+Node stack.

Q S3 + CloudFront for a React SPA — what's the one config people forget?

SPA client-side routing: a deep link like `/orders/42` has no matching S3 object → 403/404. Fix by routing error responses back to `index.html` (200) so React Router takes over. Also: cache hashed assets long, but keep `index.html` short-cached so deploys propagate.

Q IAM in one breath — roles vs users vs policies?

Policies are JSON permission documents. **Users** are long-lived identities (people). **Roles** are assumable identities with temporary credentials — what services and Lambdas should use. Principle: **least privilege**, and prefer roles over hard-coded keys.

Git & CI/CD

Listed in the stack — expect practical workflow questions, not command trivia.

Q merge vs rebase — when and why?

Merge preserves true history with a merge commit — safe for shared branches. **Rebase** replays commits onto a new base for a linear history — great to tidy a local feature branch before a PR. **Golden rule:** never rebase a branch others have pulled — it rewrites shared history.

Q What actually happens in a CI/CD pipeline?

CI: on every push — install, lint, type-check, test, build; fail fast so broken code never merges. **CD:** on merge to main — deploy the artifact to staging/prod, often with automated rollback. The point is a repeatable, human-error-free path from commit to production.

Q How do you keep secrets and env config out of Git?

`.env` in `.gitignore`; inject real values from the CI/CD secret store or AWS Secrets Manager / SSM at build or runtime. Never commit credentials — and if one leaks, rotate it, don't just delete the commit (history persists).

AI · LLMs · MCP · Agents

DIFFERENTIATOR

Heavily emphasized: "AI-driven solutions, agent-based architectures (MCP, AI agents, orchestration)." This is where you stand out. Know the vocabulary cold.

Q What is MCP (Model Context Protocol) and why does it matter?

An **open standard for connecting LLMs to external tools, data, and systems** through a uniform interface — think "a common protocol so any model can talk to any tool" rather than bespoke glue per integration. An **MCP server** exposes tools/resources; an **MCP client** (the model's host app) discovers and calls them. It standardizes what everyone was hand-rolling as function-calling plumbing.

Why they care: the JD explicitly names MCP. Even a crisp conceptual answer plus "I've read the spec / wired up a server" puts you ahead of candidates who've only heard the acronym.

Q What is an "AI agent" vs a plain LLM call? What does orchestration mean?

A single **LLM call** is one prompt in, one completion out. An **agent** wraps the model in a loop that can **use tools, observe results, and decide the next step** toward a goal — plan → act → observe → repeat. **Orchestration** is coordinating these steps (and often multiple agents/tools/services) reliably: routing, sequencing, state, retries, guardrails.

Q What is RAG and when do you use it?

Retrieval-Augmented Generation: fetch relevant documents (usually via embeddings + a vector store) and inject them into the prompt so the model answers from *your* data instead of only its training. Use it for domain knowledge, up-to-date facts, and to cut hallucination — cheaper and faster than fine-tuning for most business cases.

Q Function/tool calling — how does it work from a developer's seat?

You describe available functions (name, params, JSON schema) to the model; it responds with a structured call — which function, what arguments. **Your code executes it** and feeds the result back for the model to continue. The model decides *what* to call; it never runs anything itself. MCP is essentially a standardized transport for this pattern.

Q Tokens, context window, temperature — the practical knobs?

- **Token** — sub-word unit; billing and limits are per token.
- **Context window** — max tokens (prompt + output) the model sees at once; overflow → truncation or retrieval.
- **Temperature** — randomness. Low = deterministic (extraction, classification); higher = creative.

Q How do you keep LLM features reliable and safe in production?

Validate/parse model output against a schema (never trust free text), constrain with tool calling, add guardrails and timeouts, handle the non-deterministic failure modes, keep a human-in-the-loop for high-stakes actions, and log/evaluate outputs. Framing LLM calls as *untrusted I/O you must validate* is exactly the pragmatic mindset the JD asks for.

Cross-Cutting & "Translate Business → Technical"

The JD stresses turning business needs into scalable solutions — expect a "how would you approach X" design chat.

Q "Translate a vague business need into a technical solution" — how do you approach it?

Clarify the actual outcome and constraints first (who, scale, latency, must-haves vs nice-to-haves); restate it back. Then pick the **simplest design that meets today's need and won't wall off tomorrow's** — call out trade-offs explicitly rather than gold-plating. Show you optimize for business value and maintainability, not novelty.

Q How do you make a solution "scalable" without over-engineering?

Statelessness (scale horizontally), offload slow work to async queues, cache hot reads, index the real query patterns, and paginate. But scale for *measured* need — premature distributed complexity is its own failure. "Design for 10×, not 1000×, and measure before you shard."

Q PHP legacy is a "plus" — what do you say if asked?

Be honest about level, then frame it constructively: you're comfortable reading and maintaining/integrating legacy code, and you'd approach a legacy PHP system by understanding its boundaries and integrating cleanly (e.g. an API seam) rather than risky rewrites. They flagged it as maintenance/integration, not greenfield — match that framing.

Q Where do you put shared types between a TS frontend and Node backend?

A shared package/module (monorepo workspace) exporting the types both sides import — one source of truth for API contracts, so a backend shape change surfaces as a frontend compile error. Bonus points: deriving types from a schema (e.g. Zod) so runtime validation and static types stay in sync.